



Scalable and Accurate Parallel Timing Simulations with *parti-gem5*

José Cubero-Cascante¹ · Niko Zurstraßen¹ · Jörn Nöller¹ ·
Rainer Leupers¹ · Jan Moritz Joseph¹

Received: 12 April 2024 / Accepted: 4 April 2026
© The Author(s) 2026

Abstract

Detailed timing models are indispensable tools for the design space exploration of Multiprocessor Systems on Chip (MPSoCs). As core counts continue to increase, the complexity in memory hierarchies and interconnect topologies is also growing, making accurate predictions of design decisions more challenging than ever. In this context, the open-source Full System Simulator (FSS) *gem5* is a popular choice for MPSoC design space exploration, thanks to its flexibility and robust set of detailed timing models. However, its single-threaded simulation kernel severely hampers its throughput. To address this challenge, we introduce *parti-gem5*, an extension of *gem5* that enables parallel timing simulations on modern multi-core simulation hosts. Unlike previous works, *parti-gem5* supports *gem5*'s timing mode, the O3CPU, and Ruby's custom cache and interconnect models. Compared to reference single-thread simulations, we achieved speedups of up to 42.7× when simulating a 120-core ARM MPSoC on a 64-core x86-64 host system. While our method introduces timing deviations, we show that these can be minimised by setting a quantum based on the latencies of the target system. A quantum value just below the hit latency of the shared cache leads to an error in simulated time below 5%. Using this setting, *parti-gem5* can deliver reliable key activity indicators such as power states and operation counters for CPU, cache and DRAM modules. The combined Mean Absolute Percentage Error (MAPE) for these metrics is 2.62%.

Keywords MPSoC design space exploration · PDES · *gem5* · Ruby

✉ José Cubero-Cascante
cubero@ice.rwth-aachen.de

¹ Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany

1 Introduction

Multiprocessor Systems on Chip (MPSoCs) are present in a broad range of platforms, from web servers to automotive control units and smartphones. In addition to many cores, these systems include complex memory hierarchies with multiple levels of coherent caches and custom interconnects. System architects require detailed timing models to study the impact of hardware design choices on the overall system performance. Full System Simulators (FSSs) are a powerful tool to accomplish this task.

Among these, gem5 [1] is an open-source system-level simulator widely used in industry and academia. It has been actively maintained and developed for more than thirty years [2] and supports all major Instruction Set Architectures (ISAs) as targets, including ARM, x86 and RISC-V. gem5 is based on a Discrete Event Simulation (DES) and includes detailed models for CPU cores, caches, memory, and interconnect.

Two main simulation modes are available for modelling memory transactions. The atomic mode supports modelling functional behaviour only, and its usage is limited to software tests or quickly advancing simulations to a Region of Interest (ROI). The timing mode covers both functional and timing behaviour. With the timing mode, it is possible to study the performance impact of the interconnect, such as queuing delays or resource contention. Optimisations like pipelining and out-of-order execution can also be modelled. Hence, only the timing mode can be employed for accurate micro-architecture exploration.

One significant problem of gem5 is its single-threaded DES kernel, which limits the performance and scalability of the simulations. Since the end of Dennard scaling, much of modern computers' performance gains have been achieved due to massive parallelism enabled by more computing cores. Inherently sequential simulators only saw a single-digit relative performance increase by new CPU generations. Ultimately, the sequential performance gains cannot keep up with the increasing workload imposed by ever-larger simulation targets. This issue is even more severe when the target is a multi-core system. Based on our experiments, the timing mode achieves between 0.01 and 0.1 Mega Instructions Per Second (MIPS) on a recent high-performance workstation. If the target system is a modern MPSoC capable of achieving 64,000 MIPS, one second of the target's time would require between 1 and 10 days of host time.

Table 1 summarises the capabilities of gem5's main CPU models, their timing detail level, and their customisation support. A previous contribution, *par-gem5* [3], extended gem5 to support parallel simulations but its usage is limited to the atomic mode.

Parallel simulation support for gem5's detailed CPU models was introduced in *parti-gem5* [4]. In particular, *parti-gem5* adds support for the detailed *MinorCPU* and *O3CPU* models. It also introduces a thread-safe message-passing mechanism that enables the modelling of coherent MPSoCs using the *Ruby* [5] memory system.

Compared with the SAMOS XXIII paper [4], this journal extension broadens the evaluation of *parti-gem5* scalability and accuracy in three ways:

Table 1 Main CPU models in gem5 and their timing features

CPU model	KVM	Atomic	Minor	O3
Pipeline	N/A	None	In-order	Out-of-order
Communication protocol	N/A	Atomic	Timing	Timing
Custom cache protocols (Ruby)	✗	✗	✓	✓
Custom interconnect (Ruby)	✗	✗	✓	✓
Parallel simulation	gem5 [1, 2]	par-gem5 [3]	<i>parti-gem5</i> [4]	<i>parti-gem5</i> [4]

- We add NPB [6] workloads, increasing the benchmark set to a total of 16 parallel programs.
- We include a scalability study that compares target and host throughput in terms of IPS.
- We provide an accuracy analysis of key CPU, cache, and DRAM performance counters.

Our experimental results show speedups of up to 42.7 while maintaining a combined Mean Absolute Percentage Error (MAPE) of 5.13% for key CPU, cache, and DRAM performance statistics.

The rest of this paper is organised as follows: A summary of related work is presented in Sect. 2. Section 3 explains the key concepts pertaining to gem5 timing simulations. Section 4 presents the challenges for parallelising timing simulations and our approach to tackling them. We show our experimental evaluation of *parti-gem5* regarding performance and accuracy in Sect. 5. Finally, we conclude in Sect. 6.

2 Related Work

Parallel Discrete Event Simulation (PDES) and its application to FSS have been active areas of research for several decades with fundamental theoretical work laid in [7–9]. When distributing a simulation among several threads, consistently advancing time becomes a major challenge. In this regard, simulators can be classified as either synchronous or asynchronous [10].

In asynchronous simulators, each thread keeps track of the times of all other threads and is allowed to advance its own time if it stays within a maximum look-ahead distance t_{la} to the slowest thread. This approach is used in *Manifold* [11] and *SCode* [12].

By contrast, synchronous simulators use global synchronisation mechanisms. Among these, delta-cycle simulators, such as [10], enforce synchronisation at every single timestamp. However, limited performance gains are observed in typical FSS scenarios as the amount of simultaneous activity is often not high enough to overcome the synchronisation overhead [12].

Quantum-based protocols [13–17] are the base for another type of synchronous simulators. The simulation time is divided into windows of length $t_{q\Delta}$, called quanta, in which threads can advance independently. Synchronisation is only performed at global barrier events, leading to higher speedups. *WWT* [13], *Sniper* [14] and *HORNET* [17] are examples of window-based parallel simulators for coherent shared memory systems. In these works, the inherent parallel structure of the target multi-core system is exploited to partition the simulation workload. One host thread is created per simulated CPU core. Private caches are assigned to the same thread of their corresponding CPU. The main challenge is to handle the memory transactions which cannot be satisfied by the private caches. In *WWT*, the host directly executes all instructions and memory references that hit the private caches. Memory references that miss in the private caches are trapped and handled by a module that simulates the cache coherence protocol. *Sniper* uses a technique called interval simulation based on a mechanistic analytical model. The foundation of the model is that miss events (branch mispredictions, cache and TLB misses) divide the smooth streaming of instructions through the pipeline into so called intervals. Hornet is parallel simulator targetting multi-core systems which sets the main focus on modelling the Network on Chip (NoC).

In asynchronous and window-based synchronous approaches, each thread has a local time. This poses the risk of causality errors as there is no guarantee of a chronological execution order for the events. To prevent these errors, most PDES implementations fall back to either conservative or optimistic methods [8]. With optimistic methods, such as [18], causality errors are not prevented in the first place but are detected and corrected subsequently. Conservative methods, as in [15, 16] prevent causality errors by imposing constraints on the models' behaviour. However, some simulators [11, 12] employ domain-specific timing information and trade-off minor timing deviations against simulation speed.

In the scope of gem5, some efforts have been made to enable parallel simulations. Parallel support for KVM was integrated in 2013 to mainstream gem5 [2]. The authors from [19] introduce a parallel execution mode called *pFSA* which employs Kernel Virtual Mode (KVM) [20] to fast-forward gem5 simulations to ROIs at almost native speeds. PDES is not used to simulate the ROIs, but several independent detailed simulations are run in parallel. One limitation of KVM is that it can only be used when the target and host platform match. *dist-gem5* [16] and *COSSIM* [15] are gem5 extensions targetting distributed systems. For instance, in *dist-gem5*, multiple compute nodes are assumed to communicate via Network Interface Controllers (NICs) with a known latency. A fast and perfectly accurate simulation is attained by setting $t_{q\Delta}$ smaller or equal to the NIC latency. This, however, limits the applicability of *dist-gem5* to the simulation of distributed systems connected via NIC.

The more general *par-gem5* [3] enables the parallel simulation of MPSoCs. Their quantum-based approach allows causality errors but minimises their occurrence with a carefully-chosen system partitioning and quantum setting. This extension yields a significant acceleration of gem5 full system simulations, with up to $24.7\times$ speedups reported. However, it is restricted to gem5's atomic mode, which has minimal timing details.

In *parti-gem5* [4], the work from *par-gem5* was extended to enable the parallel simulation of MPSoCs with the most detailed timing models: the in-order Minor-CPU, the out-of-order O3CPU and the Ruby coherent memory sub-system.

3 Background

Besides offering a framework for DES, *gem5* comprises a variety of hardware models for all components of a multi-core computing system. In this section, we describe the core components that facilitate detailed timing simulations and micro-architectural exploration with *gem5*, particularly the O3CPU model and the timing and Ruby protocols.

3.1 Discrete Event Simulation in *gem5*

The kernel of *gem5* is a DES engine. HW components are termed *SimObjects*, and their behaviour is modelled through events. For an event to be executed, it must first be placed in an event queue (EQ) by calling *schedule()*. Scheduled events are ordered based on the target time and priority. A simulation thread processes all events in an event queue, one at a time, starting at its head. When processed, an event can schedule, deschedule and reschedule other events. In the default single-thread DES engine, only one event queue and one simulation thread are used. This is depicted in Fig. 1a.

Synchronous de-coupled parallel simulation, as used in *par-gem5* [3], is enabled by distributing the hardware objects into N time domains. Each domain has an independent event queue and processing thread. The simulation time is divided into slices of fixed length $t_{q\Delta}$, called quanta, in which the event queues run independently and in parallel. Barrier events at each quantum border ensure synchronisation between threads. This PDES approach is shown in Fig. 1b.

As depicted, an event may attempt to schedule another which belongs to a different queue. Special handling is required in such cases as the exact current time of the target domain is unknown, and scheduling events in the past is not allowed. We refer to this situation as inter-domain scheduling. If the target schedule time is earlier than

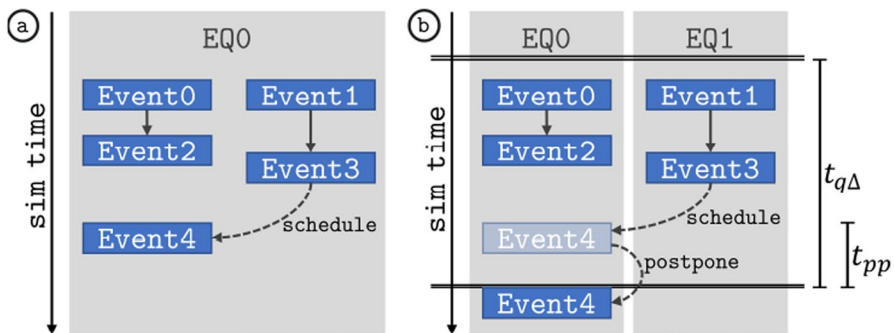


Fig. 1 Flow diagram of *gem5*'s DES and PDES. Adapted from [3]

the next quantum barrier, the event is postponed to the next quantum border. This introduces a delay $t_{pp} \in [0, t_{q\Delta}]$, which is an artefact of the parallelisation. As in [12], such timing deviations are allowed, sacrificing accuracy for speedup. However, their impact is minimised by carefully setting the quantum length and the boundaries of the simulation domains.

3.2 CPU Models in gem5

Table 1 shows the main CPU models available in gem5, which offer distinct levels of timing detail. The `KVMCPU` executes software workloads natively on the host system, based on Linux's KVM [20]. Only for specific events, e.g. when accessing a memory-mapped device, the KVM execution stops to allow simulating the corresponding device functionality. Consequently, this model can attain near-native execution speeds but does not generate any accurate statistics of the simulated platform. It should only be used to fast-forward to ROIs. The most severe limitation of KVM is its host system dependency. If the ISA of the host and the target are not the same, the KVM model cannot be used.

The `AtomicCPU` has an interpreter-like core, executing instruction after instruction with a fixed delay. It can be used to study basic cache behaviour or for software verification. In contrast to the `KVMCPU`, the memory system is also simulated, enabling the assessment of simple statistics like cache hit/miss rates.

The in-order `MinorCPU` and out-of-order `O3CPU` provide the most detailed simulation. These models include micro-architectural features like pipelining and memory access re-ordering. The authors in [21] show that these models provide excellent accuracy compared to real ARM MPSoCs. In their experiments, the error in total execution time lies between 1% to 17% for the `MinorCPU` and between 2% to 16% for the `O3CPU` with average errors of 7.4% and 8% respectively. Furthermore, these models can be combined with the highly detailed Ruby Cache system, which enables accurate modelling of cache coherence protocols and custom interconnects.

3.3 Communication Protocols in gem5

In gem5, `SimObjects` communicate via ports using packets, structures containing a target address, payload data, header and packet delay. The way packets are transmitted is determined by the selected communication protocol. Hereby, gem5 offers three protocols: functional, atomic and timing. The functional protocol is used only for debugging purposes, so we refrain from further explanation.

The atomic protocol is depicted in Fig. 2a. Transmitting a packet is achieved in one single event. The requester only needs to call `sendAtomic(pkt)`, which is received as `recvAtomic(pkt)` at the responder's side. All effects that might arise from this transaction, including coherence snoops, are handled in this single call chain. To achieve some notion of time, the function call returns a transaction delay. The requester then has to consider the delay, for instance, by scheduling the subsequent request at the current time plus the delay.

Completing a transaction in one call chain has two significant advantages: simplified code and high performance. Yet the high abstraction level comes at the price of

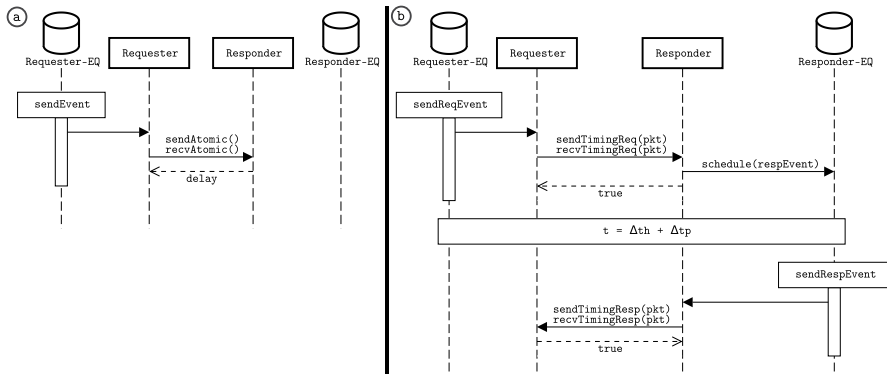


Fig. 2 Communication between a requester and a responder using the atomic mode (a) and the timing mode (b)

accuracy and limited modelling possibilities. For example, the atomic protocol does not allow the modelling of busy targets or outstanding transactions.

The timing protocol divides a transaction into two phases, implemented in separate events, as shown in Fig. 2b. The requester starts the transaction by calling `sendTimingReq(pkt)`, which is received as `recvTimingReq(pkt)` at the responder's side. If the responder accepts the request, it schedules a response event and returns `true`. During the response event, the responder calls `sendTimingResp(pkt)`, which is received as `recvTimingResp(pkt)` at the requester's side. Finally, the requester accepts the response by returning `true`. Between the request and response events, the simulation time usually advances by $\Delta t_h + \Delta t_p$, with Δt_h being the header delay and Δt_p the delay of the packet. If a requester or responder is busy receiving a transaction, it can reject the packet by returning `false`. The rejecter is responsible for signalling a retry once it is free again. In such cases, more than two simulation events per transaction are required.

Due to this higher complexity, the timing protocol has a lower performance. In our experiments, simulations using the timing protocol and the detailed O3CPU yield only 20% of the performance obtained with the atomic protocol and the AtomicCPU.

The approach of having a simple, less accurate single-phase communication and a more detailed two-phase communication protocol is also found in other simulation frameworks. The SystemC TLM-2.0 standard [22] defines a similar *blocking* transport and an *non-blocking* transport, which correspond to the atomic and the timing protocol respectively [23].

3.4 The Ruby Cache and Interconnect Subsystem

In gem5's timing mode, two cache systems are available: classic and Ruby [5]. The classic cache system provides only basic capabilities with a single pre-defined coherence protocol and a single network topology.

Ruby, on the other hand, provides highly configurable cache and interconnect models [5]. Cache transactions are simulated with high fidelity [2]. Many network topologies and cache coherence protocols are provided, and the user can define new

ones using the Specification Language for Implementing Cache Coherence (SLICC). Support for the ARM AMBA CHI protocol [24], widely used in modern high-performance ARM-based platforms, is available.

The Ruby subsystem consists on a set of interconnected nodes which communicate using a buffered message-passing protocol. Sending a message between two Ruby nodes involves three main objects: the sender node, a `MessageBuffer` and the receiver node or `Consumer`. The sender node represents the Ruby object that initiates or forwards the message. The `MessageBuffer` models the communication link; it contains a priority queue structure, which holds all messages in transit, ordered according to their insertion time. The receiver node implements the abstract class `Consumer`. Specialisations of this class, such as cache controllers and network routers, implement their specific behaviour through the virtual method: `wakeup()`. Several message buffers can be assigned to the same `Consumer`, forming an $N : 1$ relationship.

Figure 3 shows the relation between these three objects. Transmitting a message from the sender to the `Consumer` requires two phases corresponding to separate simulation events. In the first event, the sender node enqueues a new message in the `MessageBuffer`, providing a timing annotation Δ . This corresponds to the difference between the current time and the arrival time for the message. The `enqueue()` method leads to the (re-)scheduling of a `wakeup` event on the assigned `Consumer` object. During the `wakeup` event, the `Consumer` receives the message using the `dequeue()` function. All buffers assigned to this `Consumer` will be checked in search for messages ready to be received at that point in time.

One weakness of Ruby stems from its independent development. Most models, including the main transaction initiators, e.g. CPUs, GPUs and PCI modules, lack native support for the Ruby protocol. The same is true for the primary transaction targets such as the DDR controller. Hence, simulating full systems with Ruby requires

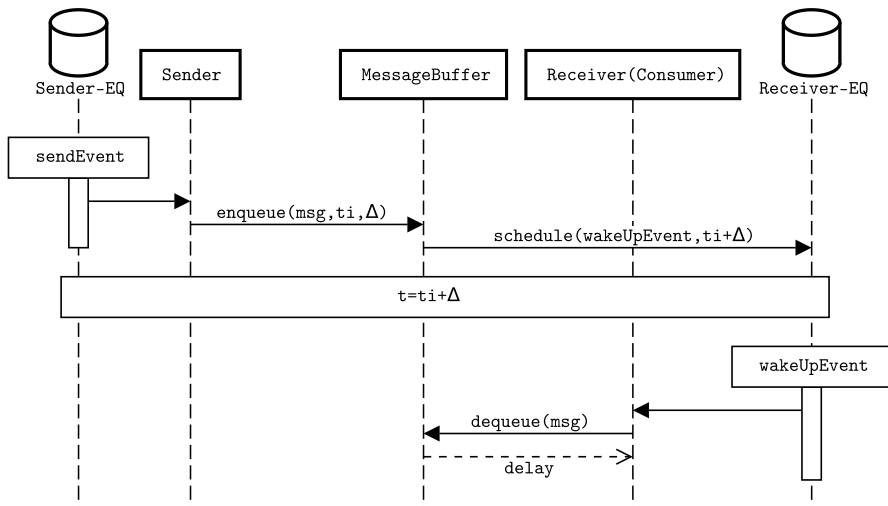


Fig. 3 Ruby message passing

constant conversions between timing protocol packets and Ruby messages. This task is performed by the *Sequencer* modules.

A typical multi-core system modelled with Ruby is depicted in Fig. 4. Ruby nodes are used to model the caches and the coherent transactions, while the timing protocol is used for non-coherent transactions. Non-coherent targets, like low-speed system peripherals and timers, are made available to the CPUs through a single input/output crossbar (*IO-XBAR*). In Fig. 4, the colour of each module indicates their supported protocol: Black modules like the CPU and the *IO-XBAR* use the gem5 timing protocol, while blue objects like the interconnect routers use the Ruby protocol.

4 Parallelising gem5 Timing Models

Due to the diversity of co-existing models and coding styles in gem5, different thread-safe techniques had to be applied. This section presents the main challenges and our solution for parallelising gem5 timing models. Our work is based on gem5 version 21.1.0.2.

4.1 System Partitioning and Main Parallelisation Challenges

As explained in Sect. 3.1, simulating in parallel requires partitioning the target system into time domains. All objects in each domain share one simulation thread and one event queue. Objects communicating within one domain have their send and receive events handled by the same thread, so their timing remains unaltered. On the contrary, sending packets across domains requires inter-domain scheduling, and timing deviations might occur. Hence, the main goal of partitioning is to minimise the number of data transactions crossing the border between event queue domains.

Our target system is arranged in a hierarchy of *SimObjects*, mirroring the modelled MPSoC structure. Following the approach from [3, 13, 14], we leverage this pre-defined hierarchy to define the time domains. Each CPU core is allocated to one distinct domain. Exclusive resources such as private caches, Transaction Lookaside Buffers (TLBs) and local interconnect are placed in their CPU's domain. This way, transactions hitting the private caches are completed within the same domain, allow-

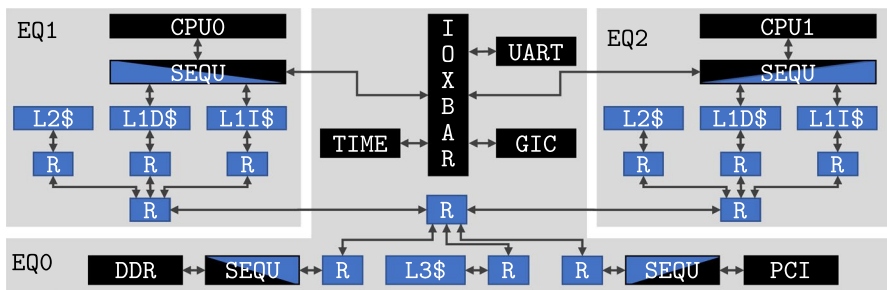


Fig. 4 Topology and event queue assignment of an exemplary Ruby system. For the sake of simplicity, some components, like message buffers or TLBs are not depicted

ing cores to advance quickly in their execution. Shared resources, such as global interconnect, global caches, main DDR memory and system peripherals, are placed in one additional domain. With this approach, $N + 1$ threads are needed for a total of N simulated CPUs.

Figure 4 shows how a two-core system is partitioned into three time domains. Two interconnect links cross the border between each CPU domain and the shared domain EQ0. The link between local and central router objects employs the Ruby protocol, while the link between the CPU's sequencer and the common *IO-XBAR* uses the timing protocol. Ensuring the correct behaviour for transactions passing through these links poses a significant challenge. The following sections present the concepts applied at these two critical interfaces to enable a thread-safe parallel simulation.

4.2 Thread-safe Ruby Message Passing

The Ruby protocol has inherent properties that can be leveraged for parallel execution. It follows a producer-consumer pattern in which the message buffer is the only shared structure. After partitioning the system, some Ruby links have sender and receiver nodes residing in different domains. Thread safety requires protecting the buffers with a mutual exclusion strategy. However, as explained next, using an independent mutex to protect each buffer is insufficient to ensure proper communication between Ruby nodes.

Some consumer objects receive messages from several senders, which might belong to several domains in a parallel simulation. This situation is depicted in Fig. 5a, where senders $S0$ and $S1$ have $C0$ as common consumer. In some cases, sender objects check the state of their output buffers before inserting a new message. For example, the sender might use the number of available slots to decide if the link is available. During the wakeup event, the consumer object checks all its input buffers, one at a time, and triggers the necessary actions to handle the incoming messages. Hence, it is necessary to guarantee a serial execution of the consumer wakeup event and the events from all linked senders.

We realise this with a shared mutex concept, illustrated in Fig. 5a. Each consumer gets a unique wakeup mutex object during initialisation. All input message buffers

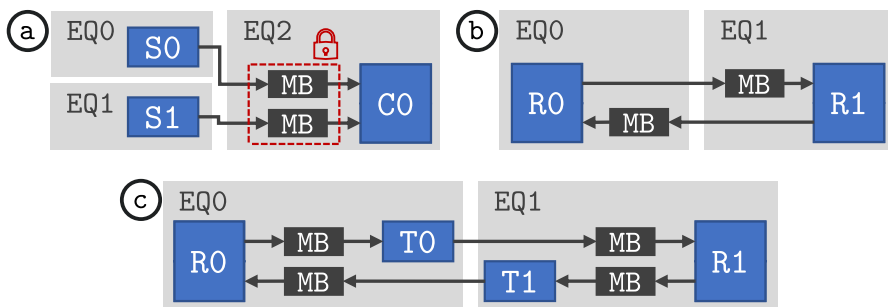


Fig. 5 Ruby parallelisation challenges and solutions. (a) Multiple senders $S0$ and $S1$ communicate with a single consumer $C0$. (b,c) Bi-directional message passing between two routers $R0$ and $R1$. The circular wait in (b) is eliminated in (c) by introducing the Throttle objects $T0$ and $T1$

assigned to this consumer share this same mutex instance, depicted as a single red dash bounding box in Fig. 5a. The consumer locks the mutex during its wakeup event, preventing the arrival of new messages to any of its input buffers. When done, the consumer releases the wakeup mutex, allowing the senders to enqueue new messages serially.

Another situation to be tackled is bi-directional communication. Routers, for example, act both as sender and consumer and must be able to communicate with each other. Such a topology could be implemented by using one message buffer for the forward link and another for the backward link, as shown in Fig. 5b. However, this configuration creates a circular wait which might result in a deadlock, a problem discussed in [8]. If the wakeup events of R0 and R1 occur simultaneously, both routers will lock their input buffers and wait indefinitely for the availability of their output buffers.

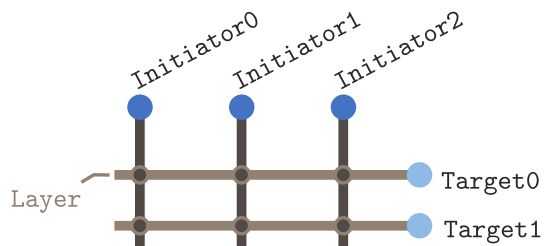
A deadlock-free implementation is created by adding one object and one message buffer to the outputs of each router. See Fig. 5c. With this arrangement, the domain border is crossed by two independent uni-directional links: T0-R1 and T1-R0. The objects T0 and T1 represent instances of the Ruby class `Throttle`. Throttle objects, placed at the output of each router, are used to model the link's bandwidth. Our parallelisation approach profits from this separation of concerns between the router and the throttle classes.

Passing messages between two Ruby nodes in different domains involves the inter-domain scheduling of the consumer's wakeup event. To ensure time consistency, the message's ready time and the wakeup event's time are adjusted during insertion to point to the next quantum border. It is important to note that postponing messages leads to over-estimation of link latencies. This is the primary source for deviations in the statistics in parallel simulations with *parti-gem5*.

4.3 Thread-safe Concurrent Non-coherent Traffic

The non-coherent Input–Output Crossbar *IO-XBar* is an N-to-M network node allowing the CPU cores to send transactions to peripherals, such as timers or input–output devices. Concurrent transactions involving two disjoint sender–receiver pairs are possible. However, concurrent transactions from two different senders to the same target must be serialised. This is accomplished using the concept of layers. A layer is a communication channel to one target and can only be occupied by one initiator at a time (see Fig. 6). Multiple layers for each phase exist since responses and requests are processed asynchronously.

Fig. 6 Example of a 3x2 non-coherent IO-crossbar in *gem5*



Whenever an initiator wants to send a message to a target, it first tries to occupy the corresponding layer. If another initiator already holds the layer, further requests are rejected. Once an initiator has claimed the layer, it can communicate with the target using the *gem5* timing protocol. Every initiator bears the responsibility to release the layer after a communication. This release is accomplished by scheduling a so-called release event, eventually informing rejected initiators to retry their transmissions.

In *parti-gem5*, multiple CPUs can compete for a layer at the same host time, even if their local simulated times differ. From the simulation time perspective, these are not concurrent accesses. However, when not adequately handled, race conditions arise. Our solution treats this situation as a special case of concurrent transactions. We extend the layer concept by protecting the crossbar layer's state with mutexes and rejecting incoming transactions if the mutex has already been locked. This slight adaptation introduces thread safety to the existing occupy and the retry mechanisms.

5 Experimental Evaluation

We evaluated the performance and accuracy of *parti-gem5* compared to the standard single-thread *gem5*. For this purpose, we simulated 16 multi-thread applications running on a scalable multi-core ARM-based target system. Our host platform is powered by an AMD Ryzen 3990x (64 dual-thread x86-64 cores), has 128GiB of 3200MHz DDR4-DRAM and runs Ubuntu Linux 22.04.

5.1 Setup

5.1.1 Target Platform

Our target is a scalable ARM-based MPSoC platform. We use the *O3CPU* to model the ARM cores and *Ruby* for the cache sub-system. To make our results comparable, we use the ARM CHI configuration provided in *gem5* as the base. Table 2 shows its most relevant features. The coherence protocol is ARM AMBA CHI [24], and the interconnect follows the hierarchical star topology shown in Fig. 4. We partition the system as described in Sect. 4.1.

The latency of links, routers and cache accesses is considered for defining a meaningful range for the simulation quantum. In our configuration, travelling from the L1 cache router to the L2 cache router and then to the L3 cache router and returning through the same path involves crossing ten links for a total of 5ns. If we add the cache access latencies from Table 2, we obtain an L3 cache hit latency of 16ns. We set this value as the maximum quantum $t_q\Delta$ to bound the link latency artefacts from the inter-domain scheduling to a reasonable range.

5.1.2 SW Workloads used for Evaluation

We selected a set of 16 SW workloads for our evaluation. The set consists of one custom synthetic benchmark *BM.SORT*, *STREAM* [25], and a sub-set of *PARSEC* [26], and *NPB* [6].

Table 2 Main characteristics of the simulated system

Component	Property	Value
CPU	Architecture	ARMv8-A 64-bit
	Clock	2GHz
L1 I-Cache	Capacity	32 KiB
	Associativity	2
	Access latency	1ns
L1 D-Cache	Capacity	64 KiB
	Associativity	2
	Access latency	1 ns
L2 Cache	Capacity	2 MiB
	Associativity	8
	Access latency	4 ns
L3 Cache	Capacity	16 MiB
	Associativity	8
	Access latency	6 ns
DRAM	Clock	1GHz
	Capacity	512 MiB
NoC	Link and router latency	0.5ns
	Router buffer size	4 messages (32-bit each)

BM. SORT¹ consists of a bare-metal multi-core test program designed to maximise CPU core utilisation while keeping the memory traffic low. Each CPU executes a sorting algorithm on an exclusive memory region. Code and data sections are kept small so all instructions and data fit within a core's private caches. There is no data sharing among threads, and the input size is scaled linearly with the number of cores.

STREAM is a synthetic benchmark that measures the sustainable memory bandwidth (in MB/s) of multi-core systems. It uses a set of four kernels representing the elemental operations on which long-vector codes are based. The kernels are specifically designed to eliminate the possibility of data re-use. Contrary to our custom benchmark, STREAM has very high cache miss rates and DDR traffic. We used the C implementation, version 5.10.²

PARSEC is a benchmark used to study the performance of MPSoCs. It includes a diverse set of applications in terms of working set, locality, data sharing, synchronisation, and off-chip traffic. We compiled PARSEC version 3.0³ for our ARM 64-bit target following the official documentation from ARM [27]. In our evaluation, six PARSEC applications are used with the `simmedium` input size setting.

NPB consists of a small set of programs designed to evaluate the performance of parallel supercomputers. We selected the eight programs from the original specification (IS, EP, CG, MG, FT, BT, SP, LU) and used the OpenMP implementation from NPB version 3.4.2.⁴ Our NPB tests use the Wide (W) data size configuration.

¹ <https://github.com/jose-cubero/gem5.bare-metal>.

² <https://www.cs.virginia.edu/stream>.

³ <https://parsec.cs.princeton.edu/download.htm>.

⁴ <https://www.nas.nasa.gov/software/npb.html>.

Table 3 Evaluated SW workloads: target and host times, simulation throughput and accuracy of *parti-gem5* with a 32-core target configuration

	gem5		parti-gem5 (mean over all $tq\Delta$)		parti-gem5 ($tq\Delta = 4ns$)	
	Target time[s]	Host time[s]	Error in target time[%]	Speedup	Error in target time[%]	Speedup
BM.SORT	2.52×10^{-3}	1.53×10^3	0.26	17.00	0.04	15.21
NPB.BT	1.60×10^{-1}	1.82×10^5	5.43	13.34	0.23	12.56
NPB.CG	5.88×10^{-2}	4.11×10^4	8.21	10.92	3.03	10.09
NPB.EP	9.31×10^{-2}	7.25×10^4	0.55	15.95	0.13	13.99
NPB.FT	1.89×10^{-2}	2.05×10^4	13.60	13.86	4.62	13.19
NPB.IS	1.33×10^{-2}	8.35×10^3	10.95	9.67	2.77	9.01
NPB.LU	5.28×10^{-1}	5.09×10^5	12.16	13.39	3.03	12.80
NPB.MG	3.82×10^{-2}	2.41×10^4	11.49	8.18	2.21	7.72
NPB.SP	6.32×10^{-1}	5.22×10^5	10.85	11.49	1.52	11.01
PARSEC.	4.38×10^{-2}	1.52×10^4	0.22	9.50	0.20	7.81
BLACKSCHOLES						
PARSEC.CANNEAL	6.79×10^{-2}	1.97×10^4	10.68	7.20	3.24	6.68
PARSEC.DEDUP	5.96×10^{-1}	8.01×10^4	8.35	3.49	2.42	3.51
PARSEC.FERRET	2.02×10^{-1}	5.38×10^4	5.19	5.75	0.87	5.59
PARSEC.	9.48×10^{-2}	5.24×10^4	5.63	11.94	4.63	11.09
FLUIDANIMATE						
PARSEC.	1.67×10^{-1}	9.28×10^4	3.62	11.45	0.36	10.37
SWAPTIONS						
STREAM	1.26	2.73×10^5	6.85	5.18	1.94	4.66
MEAN			7.13	10.52	1.95	9.70

Except for our synthetic benchmark, all applications run on top of the Ubuntu Linux 14.04 file system released by gem5.⁵ We employ fast-forwarding with the AtomicCPU and the checkpointing mechanism to simulate only the Regions of Interest (ROIs) with the detailed timing mode. Table 3 lists all the benchmarks used in our evaluation. The column target time corresponds to the duration of the ROI.

5.2 Results

5.2.1 Scalability Analysis

We perform a core sweep with BM.SORT, NPB.EP and PARSEC.BLACKSCHOLES to assess the scalability of *parti-gem5*. The number of target cores, N , is increased in multiples of 2 up to 64 cores. Additionally, we simulated with 96 and 120 target cores. Since our partitioning scheme requires $N + 1$ threads (see Sect. 4.1), we refrain from simulating with 128 cores to avoid creating more threads than are physically available. The results of this experiment are summarised in Fig. 7, where the common x-axis represents the number of target cores, N .

⁵https://www.gem5.org/documentation/general_docs/fullsystem/guest_binaries.

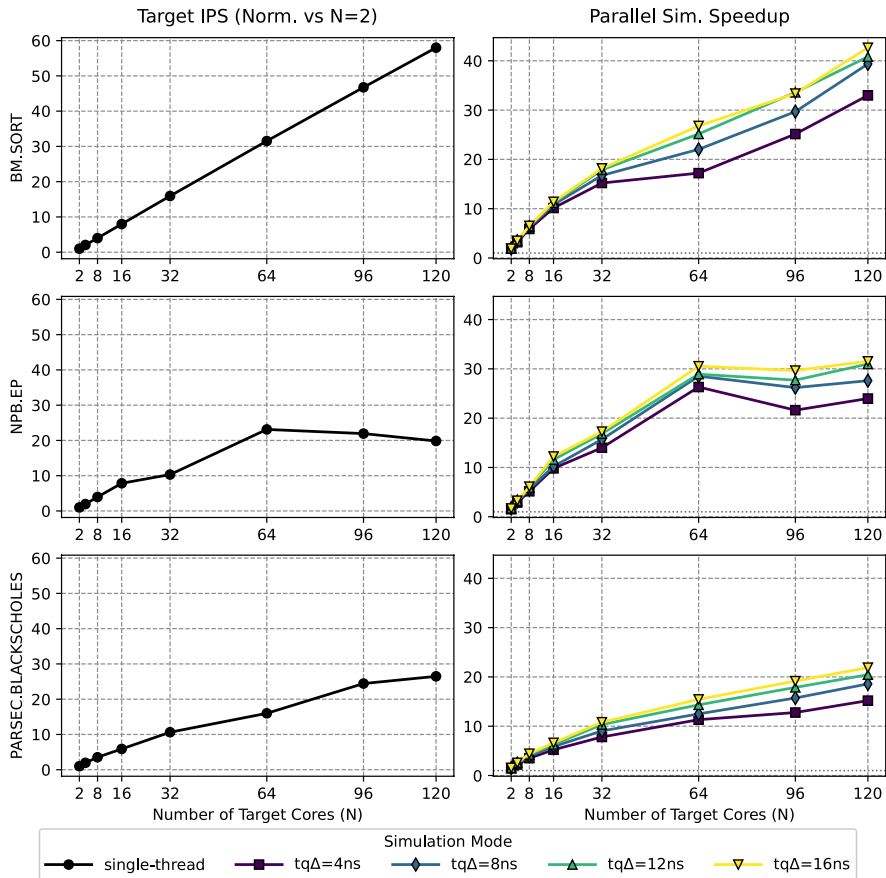


Fig. 7 Target IPS, host IPS and speedup as a function of the number of simulated cores and quantum setting.

The plots on the top row show the target IPS (target instructions per target second), obtained from a reference single-thread simulation. Our synthetic benchmark shows a perfectly linear increase, reaching a maximum of 233.3 target Giga Instructions Per Second (GIPS). `NPB. EP` has a steeper throughput gain up to 16 cores and reaches a peak throughput of 137.8 GIPS with 64 cores. However, this application sees a drop in the attainable throughput when using 120 cores. `PARSEC.BLACKSCHOLES` shows a moderate but steady growth and reaches a maximum of 69.4 GIPS.

In the middle row of Fig. 7, the host IPS (target instructions per host second) is shown. The standard single-thread simulation has a throughput ranging from 0.10 to 0.13 MIPS when simulating a 2-core system. It continually diminishes as the number of cores grows, reaching a minimum of 0.05 MIPS. As the number of cores increases, the number of simulation events associated with the cache system and interconnect becomes high. However, those events do not contribute to the host IPS metric. On the other hand, the throughput of *parti-gem5* shows a growth resembling the target system's load. `BM. SORT` attains an impressive throughput of 2.4 MIPS.

Finally, we show the performance gain of *parti-gem5* in the bottom row of Fig. 7. The plotted speedup factor is computed as the ratio between the host time of the reference single-thread simulation and the host time of the parallel simulation. The bare-metal application exhibits the highest speedups, reaching $42.7\times$ when simulating 120 cores. On the other hand, `PARSEC.BLACKSCHOLES` has the lowest gain, with a maximum speedup of $21.0\times$. This plot demonstrates that the scalability of the target SW application limits the attainable speedup gains of *parti-gem5*.

In the speedup plots, the effect of the quantum setting $tq\Delta$ is also observable. As in previous works, higher quantum values lead to an increase in speedup. The influence of this setting becomes more pronounced with higher core counts.

5.2.2 Speedup Vs Simulation Accuracy

In this section, we dig deeper into the influence of the quantum setting on speedup and accuracy. For this, we fixed the number of simulated cores, N , to 32 and run all 16 multi-thread benchmarks with 4 quantum settings: 4, 8, 12, and 16ns. We quantify the simulation error as the absolute percentual deviation in total simulated time. This is a good indicator of the overall accuracy since individual timing deviations introduced by the parallelisation will ultimately be reflected there.

The attained speedup and simulation errors are summarised in Table 3 and plotted in Fig. 8. The observed speedup covers a wide range, from $3.37\times$ for `PARSEC.DEDUP` to $18.18\times$ for `BM.SORT`. The application `NPB.EP` yields a remarkable speedup of $17.27\times$. As a result, the simulation, typically requiring almost a full day, is completed in approximately 1.15 h. On the other end, `PARSEC.DEDUP` achieves only $3.37\times$. The mean speedup is $10.52\times$. The high variation among applications confirms that the target workload throughput sets the limit for the achievable gain of *parti-gem5*.

The boxplots on the right side of Fig. 8 capture the influence of the quantum setting $tq\Delta$. Regarding speedup, increasing the $tq\Delta$ provides only slight improvements that seem to saturate. However, the mean absolute error in target time increases linearly, from 1.95% for 4ns to 13.83% for 16ns. This behaviour fits the observations made by the authors of [28]. For all considered applications, setting the quantum to 4ns leads to simulation errors below 5%.

5.2.3 Accuracy of Key Performance Counters

To demonstrate the usability of *parti-gem5* for architecture exploration and energy estimations, we show the statistical distribution of errors for key performance counters of CPUs, caches and main memory (DRAM). The data presented in this section uses the same set of simulations as Sect. 5.2.2. The selected metrics and the resulting MAPE of *parti-gem5* simulations are presented in Table 4. This representative set of statistics includes cycle counts for power states and operation counts based on type. For the CPU and cache statistics, the error is computed over the sum of counts of all 32 instances in the target platform.

For the CPU cores, we study the number of instructions and the number of active, idle and quiesce cycles. In *gem5* quiesce cycles are those in which the CPU is waiting

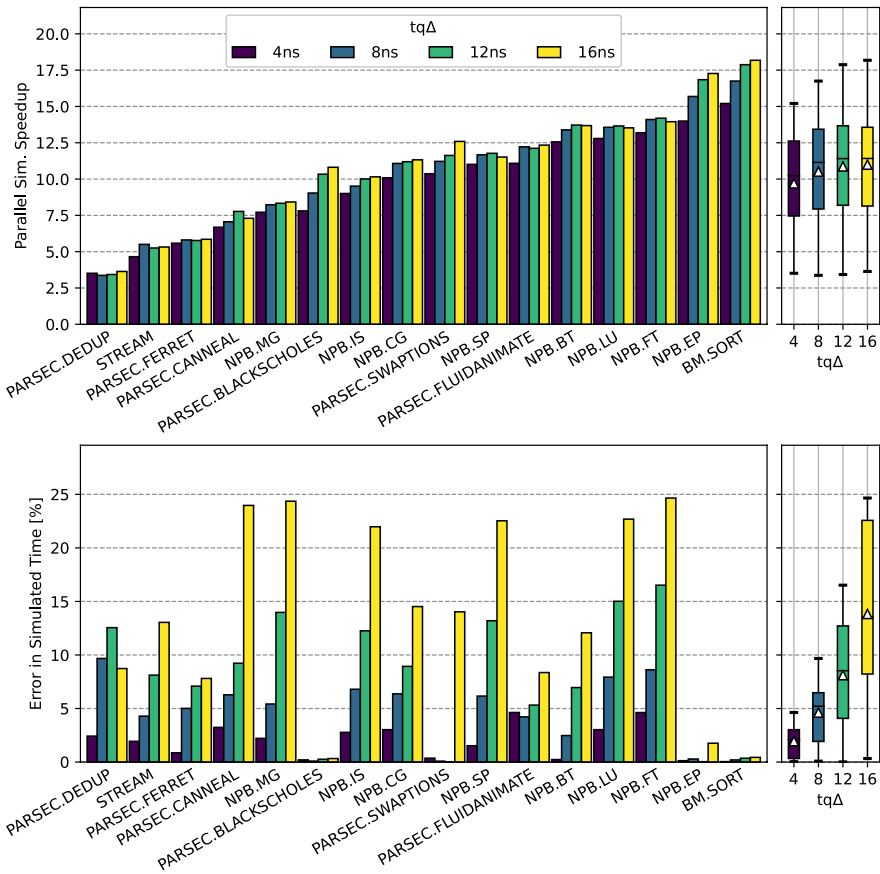


Fig. 8 Speedup and simulation error of 16 applications run on a 32-core target system. The colour of the bars indicates the quantum setting

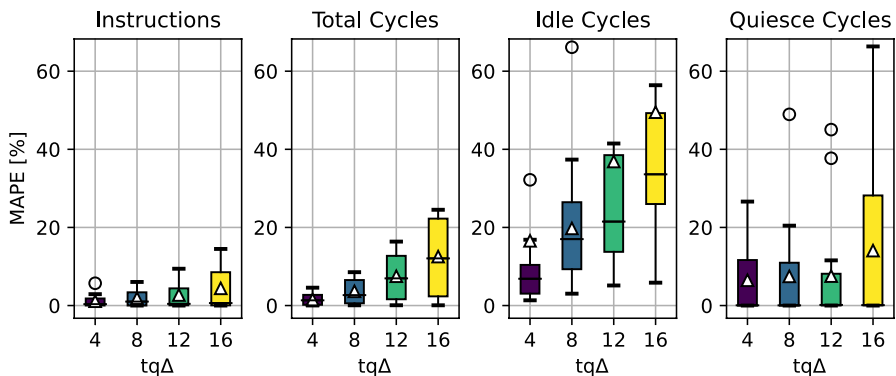
for an event or interrupt to resume execution. In the ARM architecture, this state is achieved after executing a Wait for Interrupt (WFI) or Wait for Event (WFE) instructions. Idle cycles correspond to those spent waiting due to stalls, e.g., due to a cache or TLB miss. The resulting deviations are presented in Fig. 9. The error in instruction and total cycle count is below 2%. More significant errors are observed in the idle and quiesce cycles. These two metrics are directly affected by the event postponing approach of *parti-gem5*, presented in Sect. 3.1.

In Fig. 10, the percentual errors in the number of accesses and hits to all system caches are plotted. Absolute errors below 10% for access counts and below 15% for miss counts are observable for all cache metrics and all quantum settings. The simulations using a $tq\Delta$ of 4ns show a MAPE of only 0.86%.

Finally, we present the errors in DRAM metrics in Fig. 11. As for the CPU statistics, a significant deviation occurs in the idle cycle count. The MAPE in the considered DRAM statistics is 5.7% for all $tq\Delta$ settings and 2.4% for a quantum of 4ns.

Table 4 Hardware performance counters

Domain	Statistic	MAPE (mean over all $tq\Delta$) (%)	MAPE ($tq\Delta = 4ns$) (%)
CPU	Instructions	2.53	1.11
CPU	Total Cycles	6.31	1.54
CPU	Idle Cycles	30.68	16.57
CPU	Quiesce Cycles	8.89	6.47
L1D-cache	Accesses	1.51	0.69
L1D-cache	Misses	0.53	0.35
L1I-cache	Accesses	1.23	0.34
L1I-cache	Misses	3.63	2.50
L2-cache	Accesses	0.64	0.51
L2-cache	Misses	0.85	0.65
L3-cache	Accesses	0.87	0.69
L3-cache	Misses	1.19	1.15
DRAM	Reads	0.26	0.16
DRAM	Writes	3.01	2.96
DRAM	Idle Cycles	11.23	4.26
DRAM	Active Cycles	6.74	2.62
DRAM	Refresh Cycles	7.05	1.92
MEAN		5.13	2.62

**Fig. 9** Error distribution in CPU statistics for four quantum settings

6 Conclusion and Outlook

Our work demonstrates that the acceleration of detailed timing simulations for MPSoCs on gem5 is achievable through the use of PDES, effectively harnessing the computational power of modern multi-core simulation hosts. In this context, *parti-gem5* serves as a valuable addition to existing methods, such as sampling and checkpointing, by enabling rapid exploration of micro-architectures.

One notable advantage of our approach is its ability to simulate larger portions of the target software applications with detailed models. The extent of speedup achieved relies on the scalability of the simulated multi-thread software workload.

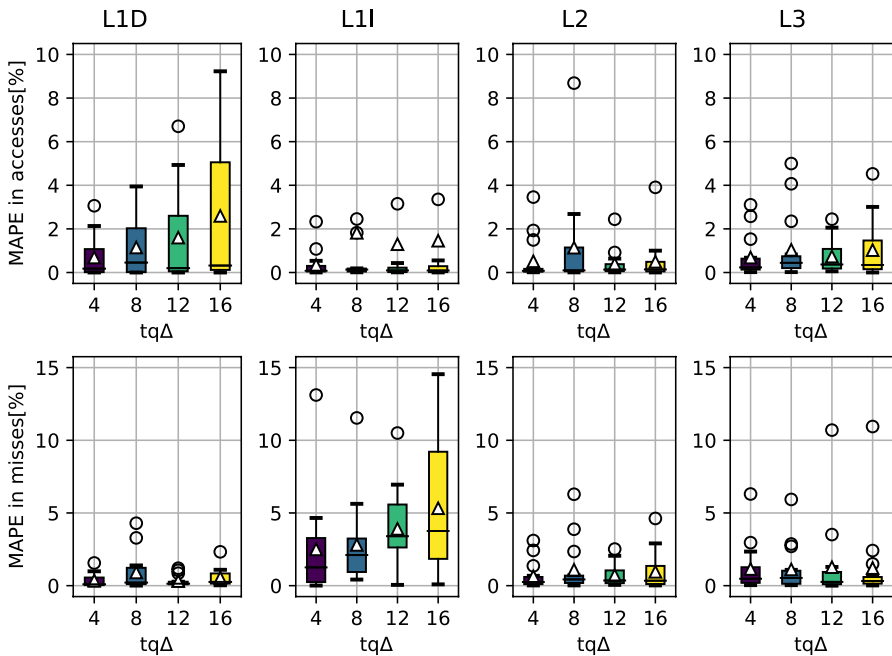


Fig. 10 Error distribution in cache statistics for four quantum settings

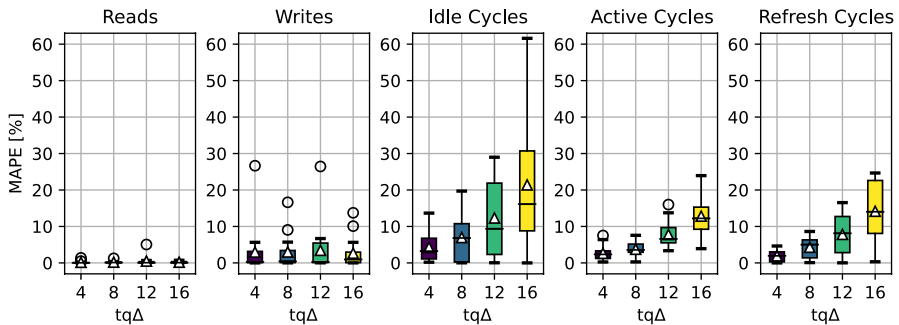


Fig. 11 Error distribution in DRAM statistics for four quantum settings

Our evaluations reveal that applications with limited data sharing greatly benefit from *parti-gem5*. Using a set of 16 benchmarks on a 32-core target platform, we showed speedups up to $18.18\times$ with a mean of $10.52\times$.

Despite the introduction of timing inaccuracies due to the non-determinism of parallel simulations, the deviations in simulated time can be bound to an acceptable range by setting a quantum-based on the latencies of the target system. We showed that for a quantum value just below the hit latency of the non-private cache (4ns in our experiments) leads to an error below 5% for all sixteen benchmarks. With this setting, *parti-gem5* can deliver reliable key activity indicators such as power states

and operation counters for CPU, cache and DRAM modules. The combined MAPE for these metrics is 2.62%.

Although we did not encounter any causality errors that affected the correctness of the simulated workloads, conducting more formal verification to validate the correctness and preservation of memory consistency would be an important contribution to our work.

While our evaluation was focused on the ARM ISA, the concepts we introduced can be extended to any other target architecture. Further experimentation is necessary to explore simulation performance for other interconnect topologies and software workloads. Nevertheless, we hope our contributions inspire and enable further advancements in this field.

New development efforts in simulation infrastructure should consider the performance benefits of multi-core host platforms early in the design process. This implies creating models and transmission protocols which are thread-safe by design.

Author Contributions J.C. wrote the main manuscript text. J.C., N.Z., and J.N. worked on the development, testing, and benchmarking of the *parti-gem5* SW. These three authors prepared all the tables and figures. R.L. is the head of the department and provided ample scientific guidance. J.J. acted as principal investigator and project manager in this project. All authors reviewed the manuscript.

Funding Open Access funding enabled and organized by Projekt DEAL.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Binkert, N., Beckmann, B., Black, G., Reinhardt, S.K., Saidi, A., Basu, A., et al.: The gem5 simulator. *SIGARCH Comput. Archit. News* **39**(2), 1–7 (2011). <https://doi.org/10.1145/2024716.2024718>
2. Lowe-Power, J., Ahmad, A.M., Akram, A., Alian, M., Amslinger, R., Andreozzi, M., et al.: The gem5 simulator: Version 20.0+. CoRR abs/2007.03152 (2020). <https://doi.org/10.48550/arXiv.2007.03152>
3. Zurstraßen, N., Cubero-Cascante, J., Joseph, J.M., Yichao, L., Xinghua, X., Leupers, R.: par-gem5: Parallelizing gem5's atomic mode. In: 2023 Design, Automation and Test in Europe Conference and Exhibition (DATE), pp. 1–6 (2023). <https://doi.org/10.23919/DATE56975.2023.10137178>

4. Cubero-Cascante, J., Zurstraßen, N., Nöller, J., Leupers, R., Joseph, J. M.: parti-gem5: gem5's timing mode parallelised. In: *Embedded Computer Systems: Architectures, Modeling, and Simulation: 23rd International Conference (SAMOS 2023)*, pp. 177–192 (2023). https://doi.org/10.1007/978-3-031-46077-7_12
5. Martin, M.M.K., Sorin, D.J., Beckmann, B.M., Marty, M.R., Xu, M., Alameldeen, A.R., et al.: Multifacet's general execution-driven multiprocessor simulator (GEMS) toolset. *SIGARCH Comput. Archit. News* **33**(4), 92–99 (2005). <https://doi.org/10.1145/1105734.1105747>
6. Jin, H., MA, F.: The OpenMP implementation of NAS Parallel Benchmarks and its performance. Technical report, MRJ Technology Solutions, Moffet Field, California, United States (jan 1999). <https://ntrs.nasa.gov/citations/20000102377>
7. Chandy, K.M., Misra, J.: Distributed simulation: a case study in design and verification of distributed programs. *IEEE Trans. Software Eng. SE-5*(5), 440–452 (1979) <https://doi.org/10.1109/TSE.1979.230182>
8. Fujimoto, R.M.: Parallel discrete event simulation. *Commun. ACM.* **33**(10), 30–53 (1990). <https://doi.org/10.1145/84537.84545>
9. Nicol, D., Fujimoto, R.: Parallel simulation today. *Ann. Oper. Res.* **53**, 249–285 (1994)
10. Schumacher, C., Leupers, R., Petras, D., Hoffmann, A.: parSC: Synchronous parallel SystemC simulation on multi-core host architectures. In: *2010 IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, 241–246 (2010). <https://doi.org/10.1145/1878961.1879005>
11. Wang, J., Beu, J., Bheda, R., Conte, T., Dong, Z., et al.: Manifold: A parallel simulation framework for multicore systems. In: *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 106–115 (2014). <https://doi.org/10.1109/ISPASS.2014.6844466>
12. Weinstock, J.H., Murillo, L.G., Leupers, R., Ascheid, G.: Parallel SystemC simulation for ESL design. *ACM Trans. Embed. Comput. Syst.* (2016). <https://doi.org/10.1145/2987374>
13. Reinhardt, S.K., Hill, M.D., Larus, J.R., Lebeck, A.R., Lewis, J.C., Wood, D.A.: The Wisconsin wind tunnel: virtual prototyping of parallel computers. *SIGMETRICS Perform. Eval. Rev.* **21**(1), 48–60 (1993). <https://doi.org/10.1145/166962.166979>
14. Carlson, T.E., Heirman, W., Eeckhout, L.: Sniper: Exploring the level of abstraction for scalable and accurate parallel multi-core simulation. In: *SC '11: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–12 (2011). <https://doi.org/10.1145/2063384.2063454>
15. Tampouratzis, N., Papaefstathiou, I., Nikitakis, A., Brokalakis, A., Andrianakis, S., Dollas, A., et al.: A novel, highly integrated simulator for parallel and distributed systems. *ACM Trans. Archit. Code Optim.* (2020). <https://doi.org/10.1145/3378934>
16. Mohammad, A., Darbaz, U., Dozsa, G., Diestelhorst, S., Kim, D., Kim, N.S.: dist-gem5: Distributed simulation of computer clusters. In: *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pp. 153–162 (2017). <https://doi.org/10.1109/ISPASS.2017.7975287>
17. Ren, P., Lis, M., Cho, M.H., Shim, K.S., Fletcher, C.W., et al.: Hornet: a cycle-level multicore simulator. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* **31**(6), 890–903 (2012). <https://doi.org/10.1109/TCAD.2012.2184760>
18. Jung, M., Schnicke, F., Damm, M., Kuhn, T., Wehn, N.: Speculative temporal decoupling using fork(). In: *2019 Design, Automation and Test in Europe Conference (DATE)*, pp. 1721–1726 (2019). <https://doi.org/10.23919/DATE.2019.8714823>
19. Sandberg, A., Nikoleris, N., Carlson, T.E., Hagersten, E., Kaxiras, S., et al.: Full speed ahead: detailed architectural simulation at near-native speed. In: *2015 IEEE International Symposium on Workload Characterization*, pp. 183–192 (2015). <https://doi.org/10.1109/IISWC.2015.29>
20. Kivity, A., Kamay, Y., Laor, D., Lublin, U., Liguori, A.: KVM: the linux virtual machine monitor. In: *Proceedings Linux Symposium* **15** (2007)
21. Endo, F.A., Couroussé, D., Charles, H.-P.: Micro-architectural simulation of in-order and out-of-order ARM microprocessors with gem5. In: *2014 International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS XIV)*, pp. 266–273 (2014). <https://doi.org/10.1109/SAMOS.2014.6893220>
22. IEEE standard for standard systemc language reference manual. In: *IEEE Std 1666-2011 (Revision of IEEE Std 1666-2005)*, 1–638 (2012) <https://doi.org/10.1109/IEEESTD.2012.6134619>

23. Menard, C., Castrillon, J., Jung, M., Wehn, N.: System simulation with gem5 and SystemC: the key-stone for full interoperability. In: 2017 International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS), pp. 62–69 (2017). <https://doi.org/10.1109/SAMOS.2017.8344612>
24. Arm Limited: AMBA 5 CHI Architecture Specification. ARM IHI 0050E.a. (2020). Arm Limited
25. McCalpin, J.D.: Memory bandwidth and machine balance in current high performance computers. In: IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter, 19–25 (1995)
26. Zhan, X., Bao, Y., Bienia, C., Li, K.: PARSEC3.0: a multicore benchmark suite with network stacks and SPLASH-2X. SIGARCH Comput. Archit. News 44(5), 1–16 (2017) <https://doi.org/10.1145/3053277.3053279>
27. Tousi, A., Zhu, C.: Arm Research Starter Kit: system modeling using gem5. https://github.com/arm-university/arm-gem5-rsk/blob/master/gem5_rsk_gem5-21.2.pdf. Accessed: 2022-01-31
28. Zurstraßen, N., Brandhofer, R., Cubero-Cascante, J., Bosbach, N., Jünger, L., Leupers, R.: The optimal quantum of temporal decoupling. In: 2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC), pp. 686–691 (2024). <https://doi.org/10.1109/ASP-DAC58780.2024.10473967>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.